XII SIINTEC
INTERNATIONAL SYMPOSIUM ON INNOVATION AND TECHNOLOGY
Currents of the future: ENERGY, CLIMATE, AND GLOBAL SUSTENTAINABILITY

UVM-Based Functional Verification of a Tightly Coupled Data Memory Interconnect for a DSP Accelerator on FPGA

Leonardo Rodrigues Soares da Conceição[*,1], **Paulo Cezar da Paixão**[1], **Nelson Alves Ferreira Neto**[1]
[1]*SENAI CIMATEC, QuIIN – Quantum Industrial Innovation, Salvador, Bahia, Brazil*
[*]*Corresponding author: SENAI CIMATEC; Avenida Orlando Gomes, 1845, Salvador – BA; leonardo.soares@fbter.org.br*

Abstract: This paper presents the methodology for the functional verification of a tightly coupled data memory (TCDM) interconnect module designed for a high-performance digital signal processing (DSP) accelerator targeting a Xilinx Zynq UltraScale+ RFSoC 4x2 field-programmable gate array (FPGA). The TCDM implements a crossbar topology connecting up to eight initiator interfaces to eight static random-access memory (SRAM) banks through parallel 512-bit data channels operating at 350 MHz. A complete verification environment was developed using the universal verification methodology (UVM), comprising multiple parallel agents, a behavioral reference model, a self-checking scoreboard, a transaction-level audit tracer for fault isolation, and a functional coverage collector. A structured test plan covering handshake protocol compliance, read and write operations, distributed arbitration under contention, address routing, continuous back-to-back traffic, and error handling was established, with primary validation focused on handshake protocol integrity and write path verification across parallel initiator agents. The verification campaign targets 90% for structural coverage metrics and 95% for functional coverage bins as acceptance criteria for the design under verification (DUV). The UVM-based environment provides the observability and controllability required to assess whether the DUV conforms to its functional specification across defined test scenarios. The results demonstrate that pairing UVM with multi-stage transaction auditing guarantees protocol compliance and cycle-accurate fault isolation for high-reliability FPGA accelerator interconnects in quantum communication systems.
Keywords: Functional verification. UVM. TCDM. FPGA. DSP accelerator.

1. Introduction

Modern field-programmable gate array (FPGA)-based digital signal processing (DSP) accelerators for continuous-variable quantum key distribution (CV-QKD) systems rely on tightly coupled data memory (TCDM) crossbar interconnects to provide low-latency parallel access between processing elements and static random-access memory (SRAM) banks [1]. Verifying such multi-master interconnects requires addressing concurrent read/write requests, arbitration contention, and protocol compliance under heavy traffic [2]. In high-throughput quantum communications, undetected interconnect deadlocks or subtle protocol violations cause catastrophic data corruption and compromise real-time processing guarantees. The universal verification methodology (UVM) [2, 3, 4] in SystemVerilog [5, 6] provides the standardized multi-agent architecture needed to meet these verification challenges.

This paper presents a UVM-based verification environment developed in Xilinx Vivado for a TCDM interconnect targeting the Xilinx Zynq UltraScale+ RFSoC 4x2 FPGA [7], supporting 8 initiators and 8 SRAM banks over parallel 512-bit data channels operating at 350 MHz. To overcome the diagnostic limitations of conventional UVM scoreboards, the environment introduces an original 8-stage audit tracer for transaction-level fault isolation alongside a behavioral reference model, self-checking scoreboard, and functional coverage closure. This setup enables thorough validation and early bug detection during simulation.

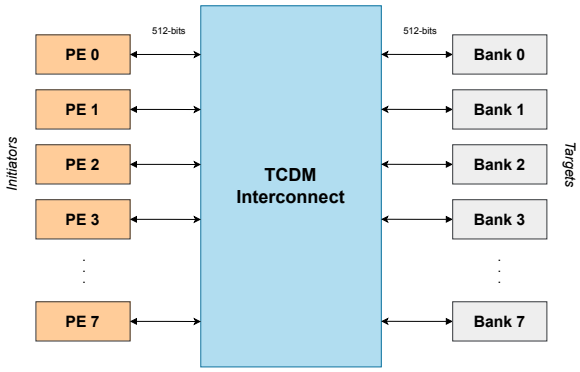
The remainder of this paper is organized as follows: Section 2 describes the TCDM architecture; Section 3 details the UVM verification envi-

ronment; Section 4 presents the verification plan; Section 5 discusses the results; and Section 6 concludes the article.

2. TCDM Architecture

The TCDM module provides a shared scratchpad memory region with low latency and high bandwidth for the DSP accelerator cluster. When all 8 processing elements (PEs) and 8 SRAM banks operate simultaneously in parallel without contention, each 512-bit PE channel operating at 350 MHz achieves 179.2 Gbps (22.4 GB/s), yielding a peak parallel aggregate throughput of 1.43 Tbps (179.2 GB/s) across the interconnect, as illustrated in Figure 1.

Figure 1: High-level block diagram of the TCDM architecture showing the interconnect between initiators and SRAM banks.



To verify the functionality of a TCDM, a verification environment must generate signals that stimulate the design under verification (DUV), reproducing the behavior of the processing elements (PEs) through the handshake protocol. The stimulated signals are presented in Table 1.

3. Verification Environment (Env)

The verification environment was developed in SystemVerilog using UVM 1.2 [3] and targets simulation with Xilinx Vivado 2025.1. The environment adopts an interface-level approach, monitoring traffic at the initiator interfaces and comparing the design under verification (DUV) responses against a behavioral reference model. Figure 2 presents the topology of the verification environment.

Table 1: TCDM Interconnect processing element signals

Signal	Dir.	Bits	Description
`clk`	In	1	System clock at 350 MHz.
`req`	In	1	Access request asserted by PE.
`add`	In	32	Target memory address (64-bytes).
`wen`	In	1	Write enable (1=Write, 0=Read).
`wdata`	In	512	Write data bus sent to crossbar.
`be`	In	64	Byte-enable write mask.
`gnt`	Out	1	Grant signal from crossbar.
`vld`	Out	1	Valid response / data ready.
`rdata`	Out	512	Read data bus returned to PE.
`err`	Out	1	Error flag on invalid access.

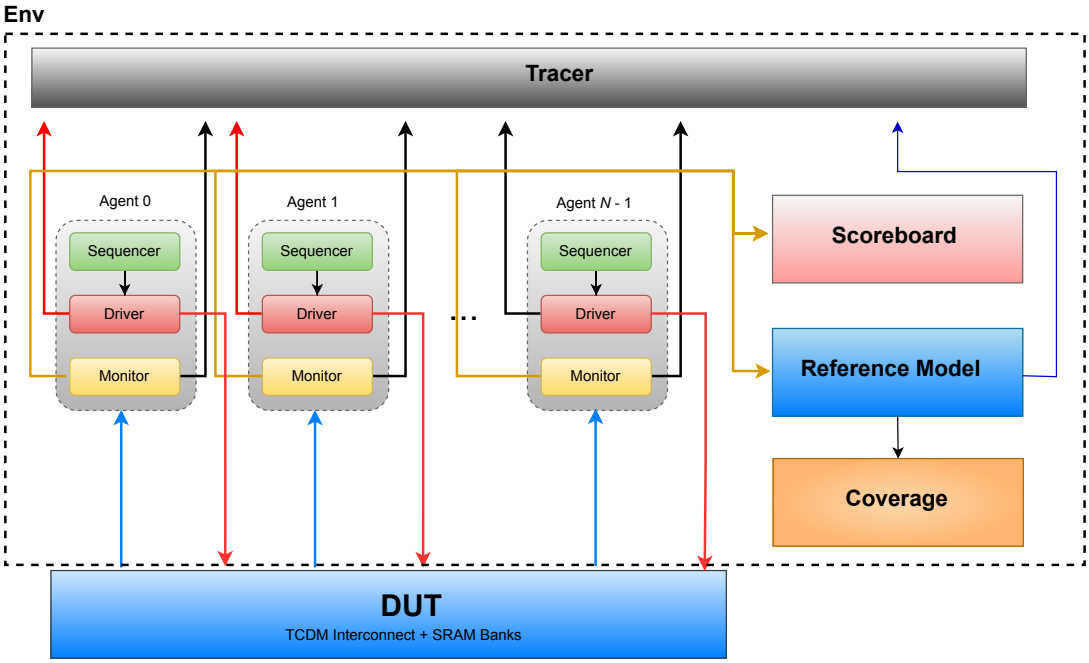
3.1. Agent Architecture

The environment instantiates N independent UVM agents (one per active PE), where N is parameterized through a compile-time define. Each agent encapsulates three components: the **Sequencer** schedules randomizable transaction objects constrained to respect 64-byte address alignment and the 512-byte PE address window; the **Driver** injects request signals, waits for `gnt`, deasserts `req`, then waits for `vld` before calling `item_done()`; and the **Monitor** observes bus signals and broadcasts each captured transaction to the reference model and the scoreboard.

3.2. Reference Model

The reference model is a purely functional component that implements the expected memory behavior without modeling protocol timing. It

Figure 2: Topology of the UVM verification environment for the TCDM module. Each PE is served by an independent agent; all agents share a single reference model, scoreboard, tracer, and coverage collector.



clones each transaction from the monitor and computes the predicted response: writes update an internal associative-array memory organized by bank, while reads return the stored value. The predicted transaction is then published to the scoreboard and coverage collector. An auxiliary debug interface mirrors these predictions as real hardware signals in the testbench top module, enabling side-by-side waveform comparison in Vivado.

3.3. Scoreboard

The scoreboard implements full-transaction comparison by field, receiving expected transactions from the reference model and actual transactions from the monitors through separate transaction-level modeling (TLM) FIFOs, organized into PE queues to preserve ordering. Each comparison checks all request and response fields individually, reporting exactly which signal diverged.

3.4. Coverage Collector

Functional coverage is collected via a UVM subscriber sampling reference model transactions, mapping directly to functional requirements (RFM-8.1 through RFM-8.7), including protocol compliance, bank routing, continuous traffic, and error handling. Arbitration under contention (RFM-8.4) is measured separately by a dedicated observer bound to the arbiter. Merged during regression testing, these complementary sampling mechanisms ensure full coverage tracking and verification closure across all operational modes.

Coverage reports are automatically generated after regression runs to identify unreached corner cases.

Table 2: Signal Propagation across the Verification Pipeline. Each row is a signal; each column is a pipeline stage. Red cells indicate where the value diverged; the Status column confirms the failing signal.

Signal	SEQ→DRV	DRV→INTF	INTF→DUT	DUT→INTF	INTF→MON	MON→RM	MON→SB	RM→SB	Status
req	1	1	1	-	-	1	1	1	ok
add	0xa0000000	0xa0000000	0xa0000240	-	-	0xa0000240	0xa0000240	0xa0000240	fail
wen	0	0	0	-	-	0	0	0	ok
be	0x00..	0x00..	0x00..	-	-	0x00..	0x00..	0x00..	ok
gnt	-	-	-	1	1	-	1	1	ok
vld	-	-	-	1	1	-	1	1	ok
err	-	-	-	0	0	-	0	0	ok
rdata	-	-	-	0xcafe..	0xcafe..	-	0xcafe..	0xcafe..	ok

3.5. Tracer

The tracer is an original audit component developed as part of this work to address a recurring limitation of conventional UVM scoreboards: when a mismatch is reported, the scoreboard identifies *what* diverged but not *where* in the verification pipeline the divergence originated.

The component receives transactions independently from three sources: driver, monitor, and reference model and aligns them per PE to perform a systematic comparison across eight propagation stages, from stimulus injection up to scoreboard evaluation. Two comparisons are performed for each matched transaction: one between the original stimulus and what the reference model received, and another between the RTL response and the predicted output. On divergence, the tracer generates the diagnostic table illustrated in Table 2, identifying the exact field and pipeline stage where the mismatch occurred, significantly accelerating root-cause analysis.

4. Verification Plan

The verification plan was derived from seven functional requirements (RFM-8.1 through RFM-8.7), each traceable to a specific aspect of the TCDM microarchitecture, as summarized in Table 3. The plan adopts a two-tier stimulus strategy: directed tests, which apply deterministic and controlled stimulus patterns to exercise each functional requirement in isolation, and a constrained-random test, which broadens stimulus coverage by randomizing address, operation, data, and byte-enable fields within the architectural constraints of the memory map. All test cases execute their sequences in parallel, with each PE driven by its dedicated agent.

Table 3: Test cases defined for the TCDM verification campaign, with traceability to functional requirements.

TC ID	Req.	Description
TC-5.1	RFM-8.1	Handshake Protocol: `gnt`, `vld`, `err` never asserted without a prior `req`.
TC-5.2	RFM-8.2	Write Verification: distinct data patterns to multiple addresses, verify delivery to correct SRAM banks.
TC-5.3	RFM-8.3	Read Verification: pre-load known data, read back and compare against expected values.
TC-5.4	RFM-8.4	Arbitration Under Contention: all PEs access the same bank; verify serialization without deadlock.
TC-5.5	RFM-8.5	Conflict-Free Parallel Routing: each PE accesses a distinct bank in parallel.
TC-5.6	RFM-8.6	Continuous Traffic: back-to-back requests from all PEs; verify stability under load.
TC-5.7	RFM-8.7	Error Handling: invalid addresses; verify safe response without protocol violations.

Each directed test case is further decomposed into

sub-tests that target specific protocol behaviors or corner cases within the requirement scope, allowing the environment to produce a consolidated per-sub-test report at the end of each simulation run.

5. Results and Discussion

Each test case was executed with 8 active PEs operating in parallel. At the end of each simulation run, the environment produces a consolidated report per test case, listing every sub-test with its individual outcome (PASS, FAIL, or SKIP) alongside the applied stimulus parameters (addresses, data patterns, byte-enable masks, and transaction counts). The following subsections detail the evaluation and execution results for each test case defined in the verification campaign.

5.1. TC-5.1 – Handshake Protocol

The handshake protocol verification began with pre-known values injected across all active PEs simultaneously, establishing a reference baseline for the protocol under ideal and deterministic conditions. Once the baseline was confirmed, randomized stimulus was applied to broaden coverage and expose protocol behaviors that deterministic patterns alone cannot fully cover.

With the baseline sub-tests validated, the campaign progressed to a series of targeted fault-injection sub-tests designed to assess the robustness of the handshake protocol under adverse and non-ideal conditions. The complete set of sub-tests executed to date, together with their individual outcomes, is presented in Table 4.

Overall, all sub-tests completed successfully with zero mismatches or protocol violations, confirming the stability and compliance of the handshake mechanism under both nominal and stress conditions. Functional coverage collection for TC-5.1 achieved full closure across all defined handshake protocol state transitions and signal assertion bins.

Table 4: TC-5.1 sub-test results: 11 PASS, 0 FAIL, 0 SKIP.

Sub-test	Description	Result
TC-5.1.A	Fixed-data basic handshake across all PEs	PASS
TC-5.1.B	Randomized-data handshake	PASS
TC-5.1.C	Stress: misaligned addresses and null byte-enables	PASS
TC-5.1.1	Grant drop mid-transaction	PASS
TC-5.1.2	Real bank contention	PASS
TC-5.1.3	Spurious grant without request	PASS
TC-5.1.4	Short `req` glitch	PASS
TC-5.1.5	Sustained back-pressure	PASS
TC-5.1.6	Rapid `req` toggling	PASS
TC-5.1.7	Ordering: `vld` never precedes `gnt`	PASS
TC-5.1.8	Reset during active handshake	PASS

5.2. TC-5.2 – Write Verification

The write path verification progressed from deterministic baseline sequences such as back-to-back writes to the same bank and address to demanding stress scenarios, including multi-PE simultaneous writes, alternating data patterns, full address sweeps, variable latency, and bank contention. Data integrity across all scenarios was automatically verified by the reference model and scoreboard pipeline, with individual sub-test results presented in Table 5.

All sub-tests completed successfully with zero data mismatches, confirming the robustness and protocol compliance of the write interface under concurrent access. Execution of TC-5.2 achieved

complete functional coverage across all specified write data patterns, byte-enable masks, and bank address range bins.

Table 5: TC-5.2 sub-test results: 7 PASS, 0 FAIL, 0 SKIP.

Sub-test	Description	Result
TC-5.2.1	Back-to-back writes to the same bank	PASS
TC-5.2.2	Address overwrite	PASS
TC-5.2.3	Simultaneous writes from multiple PEs to distinct banks	PASS
TC-5.2.4	Alternating data patterns: all-zeros, all-ones, checkerboard	PASS
TC-5.2.5	Full bank sweep across all addresses	PASS
TC-5.2.6	Variable-latency writes	PASS
TC-5.2.7	Interleaved writes from multiple PEs to the same bank	PASS

5.3. Remaining Test Cases

Test cases TC-5.3 through TC-5.7, covering read verification, arbitration under contention, conflict-free parallel routing, continuous traffic, and error handling, represent planned extension scenarios for complete regression closure. Combined, the execution of TC-5.1 and TC-5.2 successfully closed all functional coverage bins for requirement groups RFM-8.1 and RFM-8.2, establishing a solid baseline toward the 95% target across the full verification campaign. Each remaining test case follows the same methodology and rigor applied to TC-5.1 and TC-5.2: decomposition into targeted sub-tests, automated scoreboard verification, and a detailed per-sub-test stimulus record documenting the exact parameters and outcomes of each scenario.

Upon execution, these remaining test cases will finalize the functional coverage matrix, ensuring complete verification closure across all specified operational modes.

6. Conclusion

This paper presented a UVM-based verification environment for a TCDM interconnect targeting an RFSoC FPGA in CV-QKD systems. The framework validated handshake protocol compliance and write path data integrity, demonstrating that combining UVM with multi-stage audit tracing significantly reduces debug turnaround time and guarantees protocol robustness for mission-critical FPGA shared-memory subsystems. Future work focuses on executing the full suite of contention and routing test cases to achieve complete verification closure.

Acknowledgement

The authors acknowledge SENAI CIMATEC, EMBRAPII and the QuIIN (Quantum Industrial Innovation) program for providing the infrastructure and support for this research.

References

[1] Andreas Kurth. PULP and HERO Tutorial. Week of Open-Source Hardware (WOSH), 2019. Available at `https://pulp-platform.org/docs/riscv_workshop_zurich/akurth_WOSH_HERO_tutorial.pdf`.

[2] Ashok B. Mehta. *ASIC/SoC Functional Design Verification: A Comprehensive Guide to Technologies and Methodologies*. Springer, 2017.

[3] Accellera Systems Initiative. *Universal Verification Methodology (UVM) 1.2 User's Guide*. Accellera Systems Initiative, 2015. Available at `https://www.accellera.org/downloads/standards/uvm`.

[4] IEEE. *IEEE Standard for Universal Verification Methodology Language Reference Manual*. IEEE, 2020.

[5] Siemens EDA. *UVM Cookbook: Complete Verification with the Universal Verification Methodology*. Verification Academy – Siemens EDA, 2023. Available at `https://verificationacademy.com/cookbook/uvm`.

[6] IEEE. *IEEE Standard for SystemVerilog–Unified Hardware Design, Specification, and Verification Language*. IEEE, 2017.

[7] AMD. *Zynq UltraScale+ RFSoC Overview*. AMD, 2024. Available at `https://docs.amd.com/v/u/en-US/ds889-zynq-usp-rfsoc-overview`.